Sets in Python

---

1. Defining a Set

- A **set** is defined using **curly brackets {}** or the set() function.
- It does **not allow duplicate values**.
- Elements in a set are **unordered**, meaning their position is not fixed.
- It is useful when we need to store multiple values but do not want duplicates.

Example:

```
# Defining a set
my_set = {10, 20, 30, 40}
print(my_set)

# Using set() function
another_set = set([1, 2, 3, 4, 2, 3])  # Duplicates are removed
print(another_set)
```

Output:

```
{40, 10, 20, 30}
{1, 2, 3, 4}
```

---

2. Creating a Set

- A set can be created by placing elements inside {}.
- We can also create an **empty set** using set(), but {} creates an empty dictionary, not a set.

Example:

```
# Creating a set with elements
books = {"C Programming", "Java", "Python Programming"}
print(books)
# Creating an empty set
empty_set = set()
print(empty_set)  # Output: set()
```

---

3. Accessing Elements from a Set

- Sets do **not support indexing** because they are unordered.
- We can **check if an element exists** using the **in** keyword.
- We can also **loop through a set** using a for loop.

Example:

```
# Checking if an element exists
my_set = {10, 20, 30, 40}
print(20 in my_set)  # Output: True
print(50 in my_set)  # Output: False

# Looping through a set
for item in my_set:
    print(item)
```

---

4. Add and Update a Set

Adding a Single Element

- Use the .add() method to insert one element at a time.

Adding Multiple Elements

- Use the .update() method to add multiple elements from a list, tuple, or another set.

Example:

```
# Adding a single element
numbers = {1, 2, 3}
numbers.add(4)
print(numbers)  # Output: {1, 2, 3, 4}

# Adding multiple elements
numbers.update([5, 6, 7])
print(numbers)  # Output: {1, 2, 3, 4, 5, 6, 7}
```

---

5. Remove an Element from a Set

- **remove()** method: Removes a specific element; raises an error if the element is not found.

- **discard()** method: Removes a specific element; does not raise an error if the element is missing.
- **pop()** method: Removes and returns a **random** element from the set.
- **clear()** method: Removes all elements from the set.

Example:

```
# Using remove() - Raises an error if element is not found
numbers = {1, 2, 3, 4, 5}
numbers.remove(3)
print(numbers)  # Output: {1, 2, 4, 5}

# Using discard() - Does not raise an error if element is not found
numbers.discard(10)  # No error even if 10 is not in set
print(numbers)  # Output: {1, 2, 4, 5}

# Using pop() - Removes a random element
removed_element = numbers.pop()
print("Removed:", removed_element)
print(numbers)

# Using clear() - Empties the set
numbers.clear()
print(numbers)  # Output: set()
```

Summary:

✅ **Set does not allow duplicates.**
✅ **Set elements are unordered (No indexing).**
✅ **Use add() for single and update() for multiple elements.**
✅ **Use remove(), discard(), pop(), and clear() to delete elements.**